# From LaTeX to Quarto

**Plain-text writing, reproducible documents, and version control for students**

Teaching handout

2026-07-07

## Table of contents

> **Instructor note**
>
> This handout is written as a `.qmd` file on purpose. Students can read it as a normal document, but they can also open the source file in **Visual Studio Code** and see the same ideas being demonstrated by the teaching material itself.
> The main message is simple: **modern document tools are not scary if we separate source, tool, and output.**

# 1 Learning goals

By the end of this lesson, students should be able to explain:

1. Why plain-text documents are useful for writing, collaboration, and **version control**.
2. How Markdown, LaTeX, notebooks, R Markdown, and Quarto are related.
3. Why Quarto is useful for manuscripts, reports, slides, websites, and general professional documents.
4. What role **Visual Studio Code** and **Git** play in a Quarto workflow.
5. Why `.qmd` files are easier to review with **AI** tools and **version control** than binary files like `.docx` or `.pptx`.

## 2 The big idea: source, tool, output

Many students first see document tools as a confusing **list** of names:

- LaTeX
- Markdown
- Jupyter Notebook
- R Markdown
- Quarto
- **Git**
- **Visual Studio Code**
- RStudio / Positron
- Word
- PowerPoint

A better way to understand them is to ask three questions:

| Question | Example answer |
|---|---|
| **What is the source file?** | `.tex`, `.md`, `.qmd`, `.ipynb`, `.docx` |
| **What tool reads the source?** | LaTeX, Pandoc, Quarto, Jupyter, Word |
| **What output do we want?** | PDF, HTML, Word, slides, website, manuscript |

For example:

```
source file  ->  rendering tool  ->  output file
index.qmd    ->  Quarto          ->  HTML / PDF / DOCX / slides
paper.tex    ->  LaTeX           ->  PDF
README.md    ->  Markdown viewer ->  HTML preview
notebook.ipynb -> Jupyter        ->  interactive notebook / HTML / PDF
```

Student-friendly mental model

A `.qmd` file is like a recipe. Quarto is the kitchen. The final PDF, Word file, website, or slide deck is the finished meal.
If the recipe is plain text, it is easy to edit, compare, share, search, and improve with **AI** tools.

## 3 Why plain text matters

A plain-text file stores the actual content as readable characters. You can open it in almost any editor. You can compare two versions line by line. You can ask **Git** to show what changed. You can ask an **AI** assistant to edit only a section without touching formatting.

A binary file, such as a typical `.docx` or `.pptx`, is not as easy to compare directly. These files are very useful as final delivery formats, but they are less friendly as the **source of truth** for version-controlled writing.

A strong modern workflow is:

```
Write in plain text -> Track changes in Git -> Render final outputs when needed
```

For Quarto, that often means:

```
.qmd source -> Git history -> HTML / PDF / DOCX / PPTX output
```

## 4 A short history of scientific and technical writing tools

This history is not about memorizing dates. It is about understanding how each tool solved a problem created by the previous generation of tools.

### 4.1 1. TeX and LaTeX: high-quality scientific typesetting

Before modern web publishing, scientists, mathematicians, and engineers needed a reliable way to typeset complex documents, especially documents with equations, references, figures, and precise formatting.

**TeX** was created by Donald Knuth to produce high-quality typesetting, especially for technical and mathematical documents. The TeX Users Group describes TeX as the typesetting system created by Knuth, and CTAN notes that the TeX project started in 1978 while Knuth was

revising *The Art of Computer Programming* and was dissatisfied with the quality of typeset proofs.

**LaTeX** was created by Leslie Lamport as a higher-level system built on TeX. Instead of manually controlling every detail, authors could use commands such as `\section{}`, `\cite{}`, and `\begin{equation}` to describe the structure of a document.

A tiny LaTeX example:

```
\section{Introduction}

Prior work showed similar results \cite{smith2024}.

\begin{equation}
E = mc^2
\end{equation}
```

LaTeX became popular because it is excellent for:

- equations,
- automatic numbering,
- **cross-references**,
- bibliographies,
- journal and conference templates,
- high-quality PDF output.

But LaTeX can feel intimidating to new users because the source file contains many commands.

## 4.2 2. LyX: a friendlier interface to structured LaTeX writing

LyX tried to make LaTeX easier by giving users a graphical structured-writing interface. LyX describes its approach as **WYSIWYM**, or "what you see is what you mean," rather than ordinary WYSIWYG, or "what you see is what you get."

The difference:

| Approach | Meaning |
|---|---|
| WYSIWYG | You directly manipulate how the document looks. |
| WYSIWYM | You describe what the document means: title, section, theorem, equation, citation. |

LyX is useful because it helps people focus on document structure while still using LaTeX underneath.

### 4.3 3. Markdown: readable plain text for the web

Markdown appeared as a simpler way to write formatted text. Its original design goal was readability: the source should look good even before conversion to HTML.

Markdown examples:

```
# Heading

This is **bold** and this is *italic*.

- First item
- Second item

[Link text](https://example.com)
```

Markdown is easy to learn, so it became common for README files, notes, websites, documentation, and simple reports.

However, plain Markdown by itself is limited for scientific writing. It does not standardize all the things researchers often need:

- formal citations,
- numbered figures,
- **cross-references**,
- equations,
- multiple output formats,
- executable analysis code.

### 4.4 4. Pandoc: a universal document converter

Pandoc helped connect many writing formats. It can convert among many markup and word-processing formats, including Markdown, HTML, LaTeX, and Word `.docx`.

That matters because writers often want one source file but many possible outputs:

```
Markdown-like source -> Pandoc -> HTML / PDF / DOCX / slides
```

Quarto is built on top of Pandoc, so Quarto inherits much of Pandoc's multi-format publishing power.

### 4.5 5. Notebooks: writing and computation together

**Jupyter notebooks** made it common to combine code, output, plots, math, and explanatory text in one interactive document.

A notebook is useful when you want to show the process:

```
question -> code -> result -> explanation -> next question
```

This is especially helpful in data science, **statistics**, machine learning, and scientific computing.

But notebooks can also become messy:

- Cells may be run out of order.
- Output may not match the current code.
- Large `.ipynb` files can be harder to review in **Git** than plain-text files.
- A notebook can be excellent for exploration but less ideal as the final manuscript or report.

### 4.6 6. R Markdown: reproducible reports for R users

R Markdown combined Markdown text with executable R code chunks. This allowed students and researchers to write a report where the analysis and the explanation lived together.

A typical R Markdown idea:

```
narrative text + R code + output = reproducible report
```

R Markdown became popular in **statistics** and data science education because it discouraged copy-and-paste workflows. Instead of copying a graph from one program into a Word document, the graph could be generated inside the report itself.

### 4.7 7. Quarto: a modern, multi-language publishing system

Quarto can be understood as a modern successor to R Markdown that is not limited to R. It supports multiple languages and engines, including R, Python, Julia, Jupyter, and Knitr workflows.

Quarto is useful because it combines several ideas:

```
Markdown readability
+ LaTeX-style math
+ citations and cross-references
+ executable code
+ notebooks when needed
+ multi-format export
+ Git-friendly source files
```

A `.qmd` file is often the best source of truth for a modern technical document because it is readable like Markdown but powerful enough for serious publishing.

## 5 How `.qmd` compares with ordinary `.md`

A `.qmd` file is not a completely new language. It is mostly Markdown plus Quarto features.

| Feature | `.md` Markdown | `.qmd` Quarto Markdown |
|---|---|---|
| Plain text | Yes | Yes |
| Headings, lists, links | Yes | Yes |
| YAML document metadata | Sometimes | Yes, central |
| Citations | Not standard | Yes |
| Cross-references | Not standard | Yes |
| Equations | Sometimes, depending on renderer | Yes |
| Executable code chunks | No | Yes |
| Export to PDF, Word, HTML, slides | Requires extra tooling | Built into Quarto workflow |
| Manuscript/project structure | No | Yes |

A good short explanation for students:

```
Markdown is for writing simple formatted text.
Quarto Markdown is for publishing structured, reproducible documents.
```

## 6 Basic `.qmd` syntax

A Quarto document usually has three parts:

1. YAML header
2. Markdown body
3. Optional executable code chunks

## 6.1 YAML header

```
---
title: "My Report"
author: "Student Name"
format:
  html: default
  pdf: default
  docx: default
bibliography: references.bib
---
```

The YAML header tells Quarto how to render the document.

## 6.2 Markdown body

```
# Introduction

This is normal text.

This word is **bold** and this word is *italic*.

- Item one
- Item two

[Quarto website](https://quarto.org)
```

## 6.3 Equations

Inline equation:

```
The famous equation is $E = mc^2$.
```

Display equation:

```
$$
y = \beta_0 + \beta_1x + \epsilon
$$
```

Labeled equation:

```
$$
y = \beta_0 + \beta_1x + \epsilon
$$ {#eq-model}

See @eq-model.
```

### 6.4 Figures and cross-references

```
![Main result.](figures/main-result.png){#fig-main}

As shown in @fig-main, the treatment group increased.
```

Quarto labels usually use prefixes:

| Object | Label pattern | Reference pattern |
|---|---|---|
| Figure | `#fig-name` | `@fig-name` |
| Table | `#tbl-name` | `@tbl-name` |
| Equation | `#eq-name` | `@eq-name` |
| Section | `#sec-name` | `@sec-name` |

### 6.5 Citations

If your bibliography file contains an entry named `smith2024`, you can cite it like this:

```
Prior work showed a similar pattern [@smith2024].

Several studies support this view [@smith2024; @lee2022; @chen2020].
```

A simple BibTeX entry looks like this:

```
@article{smith2024,
  title = {Example Scientific Article},
  author = {Smith, Jane and Lee, Alex},
  journal = {Journal of Examples},
  year = {2024},
  volume = {12},
  number = {3},
  pages = {100--110},
  doi = {10.0000/example}
}
```

### 6.6 Code chunks

A displayed code block in ordinary Markdown looks like this:

````
```python
print("This code is shown, not executed.")
```
````

An executable Quarto code chunk uses braces around the language name:

For teaching beginners, this distinction is important:

| Syntax | Meaning |
|---|---|
| ```` ```python ```` | Display code only |
| ```` ```{python} ```` | Execute code during rendering |

## 7 Comparison of common tools

### 7.1 Quick comparison table

| Tool | What it is | Best use | Beginner difficulty |
|---|---|---|---|
| Word / Google Docs | Visual word processor | Everyday writing, comments, tracked changes | Low |
| PowerPoint | Visual slide editor | Highly designed slides | Low |

| Tool | What it is | Best use | Beginner difficulty |
|---|---|---|---|
| Markdown | Plain-text formatting syntax | Notes, README files, simple web content | Low |
| LaTeX | Technical typesetting system | Math-heavy papers, theses, journal templates | Medium to high |
| LyX | Structured graphical editor for LaTeX | LaTeX-like writing with less raw markup | Medium |
| Jupyter Notebook | Interactive computational notebook | Exploration, teaching code, data analysis | Medium |
| R Markdown | R-centered reproducible document system | R reports, statistics teaching | Medium |
| Quarto | Multi-language scientific and technical publishing system | Reports, papers, slides, websites, books | Medium |
| Git | Version control system | Tracking changes and collaboration | Medium |
| VS Code | Code/text editor | Editing `.qmd`, code, Git projects | Medium |

### 7.2 Which one should students use?

| Student task | Recommended tool |
|---|---|
| Short class notes | Markdown or Quarto |
| Homework with code and explanation | Quarto or Jupyter |
| Data analysis report | Quarto |
| Scientific manuscript | Quarto or LaTeX |
| Math-heavy journal article with mandatory template | LaTeX |
| Slides from the same source as a report | Quarto |
| Highly designed marketing slides | PowerPoint / Canva |
| Collaborative line-by-line review | Quarto + Git |
| Final document for nontechnical collaborator | Render Quarto to DOCX or PDF |

## 8 Why VS Code + Quarto + Git is a good teaching setup

In this class, we can use three separate tools, each with a clear job.

| Tool | Job |
|---|---|
| **Visual Studio Code** | Edit the source files. |
| **Quarto CLI** | Render `.qmd` files into HTML, PDF, Word, or slides. |
| **Git** | Track changes over time. |

This keeps the workflow clean:

```
edit -> preview -> render -> commit
```

A typical student workflow:

```
quarto preview
```

Then edit the file, save, and preview again.

For final output:

```
quarto render
quarto render index.qmd --to html
quarto render index.qmd --to docx
quarto render index.qmd --to pdf
```

For **version control**:

```
git status
git add index.qmd
git commit -m "Revise introduction"
```

> ⚠ Common misconception
>
> Students do **not** need to understand every tool on day one.
> A useful first-day goal is much smaller:
>
> 1. Open a `.qmd` file.
> 2. Edit a paragraph.

3. Preview the document.
4. Commit the change with **Git**.

## 9 Recommended class project structure

A small Quarto project might look like this:

```
my-quarto-project/
  _quarto.yml
  index.qmd
  references.bib
  figures/
  data/
  scripts/
  README.md
  .gitignore
```

For an undergraduate class, a simpler version may be better:

```
lesson-01-quarto/
  index.qmd
  README.md
```

Later, students can add:

```
references.bib
figures/
data/
scripts/
```

## 10 Minimal `_quarto.yml`

For a project with one or more documents:

```yaml
project:
  type: default
  output-dir: _output
```

```yaml
format:
  html:
    toc: true
    number-sections: true
  docx: default
  pdf: default

execute:
  echo: true
  warning: false
  message: false
```

For a manuscript project:

```yaml
project:
  type: manuscript
  output-dir: _output

manuscript:
  article: index.qmd

format:
  html:
    toc: true
  docx: default
  pdf: default

execute:
  freeze: auto
  echo: false
  warning: false
  message: false
```

# 11 A 15-minute classroom demonstration

## 11.1 Step 1: Show the source file

Open `index.qmd` in VS Code.

Point out:

- The YAML header at the top.
- Normal Markdown paragraphs.
- A heading using `#`.
- A math equation using `$...$` or `$$...$$`.
- A figure reference such as `@fig-example`.

### 11.2 Step 2: Render or preview

Run:

```
quarto preview
```

Explain:

```
The source is plain text.
The preview is the rendered document.
```

### 11.3 Step 3: Make a small edit

Change a title, add a paragraph, or add a bullet **list**.

### 11.4 Step 4: Show Git status

```
git status
git diff
```

Explain that **Git** can show exactly what changed because `.qmd` is plain text.

### 11.5 Step 5: Commit the change

```
git add index.qmd
git commit -m "Add introduction paragraph"
```

### 11.6 Step 6: Render another output

```
quarto render index.qmd --to docx
```

Explain that the same source can become a Word file.

# 12 Suggested student exercise

## 12.1 Exercise: Convert a paragraph into a structured Quarto document

Start with this plain paragraph:

```
This report introduces renewable energy. Solar and wind energy have grown quickly in recent y
```

Ask students to convert it into a `.qmd` file:

```
---
title: "Renewable Energy Report"
author: "Your Name"
format: html
---

# Introduction

This report introduces renewable energy. Solar and wind energy have grown quickly in recent y

# Main Questions

- What are the benefits?
- What are the limitations?
- What are the future challenges?

# Conclusion

The goal is to compare benefits, limitations, and future challenges.
```

Then ask them to preview it.

## 13 How AI fits into this workflow

**AI** tools are especially useful when the source is plain text.

Instead of asking **AI** to edit a binary PowerPoint or Word file, students can ask **AI** to edit a specific source file:

```
Edit only the Discussion section of index.qmd.
Preserve citations and cross-references.
Do not invent sources.
Keep the tone appropriate for undergraduate readers.
Return a summary of changes.
```

Good **AI** workflow:

```
AI proposes changes -> student reviews diff -> student renders document -> student commits fi
```

Bad **AI** workflow:

```
AI rewrites everything -> student cannot tell what changed -> final document contains unsuppo
```

> ! Teaching point
>
> **Version control** is not just for programmers. It is also useful for writers, researchers, analysts, policy teams, designers, and content creators.
> **Git** answers an important question: **what changed, when, and why?**

## 14 A practical recommendation

For this class, the recommended workflow is:

```
VS Code for editing
+ Quarto for rendering
+ Git for version control
+ GitHub or GitLab for sharing
```

For source files:

```
.qmd for documents
.bib for references
.png / .svg for figures
.csv for small data examples
.py / .R for analysis scripts
```

For generated outputs:

```
.html for preview and sharing
.pdf for polished reading
.docx for Word-based review
.pptx or revealjs for slides
```

## 15 What students should remember

The names are less important than the pattern.

```
Plain text source + rendering tool + version control = reproducible writing workflow
```

- LaTeX taught us that structure and typesetting matter.
- Markdown taught us that source files can be readable.
- Notebooks taught us that code, explanation, and output can live together.
- R Markdown taught us that reports can be reproducible.
- Quarto combines these ideas into a modern publishing workflow.
- **Git** makes the writing process reviewable and recoverable.
- VS Code gives students a practical environment for all of this.

## 16 Sources and suggested reading

These sources are good for instructors who want to verify the historical and technical points in this lesson.

### 16.1 Core tools

- Quarto documentation
- Quarto Markdown basics
- Quarto manuscripts
- Quarto VS Code extension documentation

- Pandoc official website
- Pandoc User's Guide

### 16.2 History and context

- Markdown by John Gruber
- Markdown syntax documentation
- TeX Users Group: What is TeX?
- CTAN: What are TeX and its friends?
- LaTeX Project: history papers
- LyX official website
- LyX 1.0.0 release announcement
- Project Jupyter: About Us
- IPython: About
- Project Jupyter: The Big Split
- R Markdown official site
- Posit announcement of Quarto
- RStudio is becoming Posit
- Git official site
- Pro Git: About version control

## 17 Optional discussion questions

1. Why might a `.qmd` file be easier to review than a `.docx` file in **Git**?
2. When would Word or PowerPoint still be the better tool?
3. Why might a notebook be good for exploration but less ideal for final writing?
4. Why does a citation manager plus `references.bib` help avoid messy reference formatting?
5. How could a marketing team, policy team, or business analyst use the same workflow even without writing scientific papers?

## 18 One-sentence summary

Quarto is a modern way to write structured documents in plain text, render them into many formats, include code and citations when needed, and track the whole process with **Git**.